

26-1 이니로 알고리즘 멘토링

멘토 - 김수성

5주차 – Trie

접두사 찾기 /14426

백준 14426 / <https://www.acmicpc.net/problem/14426>

N개의 문자열과 M개의 문자열이 주어짐

N개의 문자열에 대한 집합 S에 대해서
M개의 문자열 중에서 집합 S에 포함되어 있는 문자열 중
적어도 하나의 접두사인 것의 개수를 구해야 함

접두사 찾기 /14426

예제 입력 1 복사

```
5 10
baekjoononlinejudge
startlink
codeplus
sundaycoding
codingsh
baekjoon
star
start
code
sunday
coding
cod
online
judge
plus
```

예제 출력 1 복사

```
7
```

접두사 찾기 /14426

우선 나이브한 풀이를 생각해보자

단순 문자열 비교는 unordered map(해시)을 사용하면
 $O(1)$ 에 구할 수 있음

$O(N * L^2)$ 에 모든 접두사를 만들고 $O(1)$ 에
각 M개에 문자열에 대해서 답을 구하면 됨

-> 쿼리 $O(M)$, 삽입 $O(N * L^2)$
 $N = 10,000, L = 500$ 이므로 시간 초과

```
string a[MAX], b[MAX];
unordered_set <string> mp;
void run(){
    cin >> n >> m;
    for(int i = 1; i <= n; i++) cin >> a[i];
    for(int i = 1; i <= m; i++) cin >> b[i];

    for(int i = 1; i <= n; i++){
        string now;
        for(auto& c : a[i]){
            now += c;
            mp.insert(now);
        }
    }

    int result = 0;
    for(int i = 1; i <= m; i++){
        if(mp.count(b[i])) result++;
    }

    cout << result;
}
```

접두사 찾기 /14426

쿼리 $O(M)$, 삽입 $O(N * L^2)$

$N = 10,000$, $L = 500$ 이므로 시간 초과

삽입의 시간 복잡도를 줄이는 방법?

모든 접두사를 자료구조에 넣지 말고

접두사라는 조건에서 중복된 정보를 압축할 수는 없을까?

접두사 찾기 /14426

모든 접두사를 자료구조에 넣지 말고
접두사라는 조건에서 중복된 정보를 압축할 수는 없을까?

i
in
iny
inyr
inyro

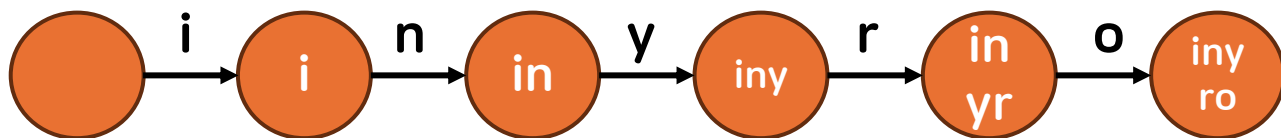
접두사를 길이 순서대로 나열하면 이전의 값에서
1개의 문자만 추가

접두사 찾기 /14426

접두사를 길이 순서대로 나열하면 이전의 값에서
1개의 문자만 추가

문자열의 정보를 그래프로 나타내 보자

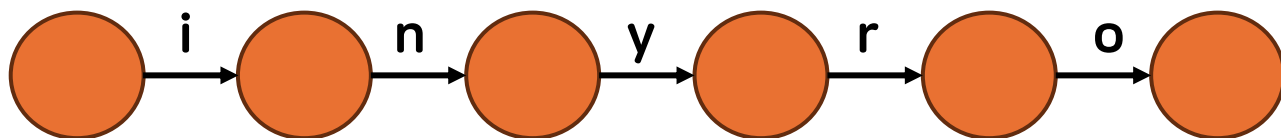
각 정점을 문자열에 간선을 추가되는 문자에 대응할 수 있음



접두사 찾기 /14426

각 정점에 접두사 자체를 저장하면
이전의 나이브한 풀이랑 시간 복잡도가 같음

-> 노드에는 문자열의 정보를 저장하지 말자

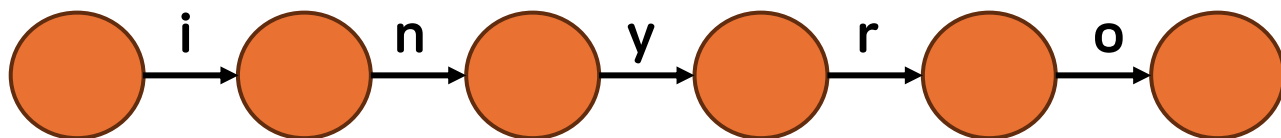


접두사 찾기 /14426

각 노드에는 문자열의 정보가 없으므로
어떤 문자열이 존재하는지 판단하기 위해서는 간선을 따라가야 함

-> 문자열 존재 판정에 $O(L)$ 의 시간 복잡도가 들음

또한 각 정점은 삽입한 문자열의 접두사이므로
접두사의 존재 판정도 같이 할 수 있음

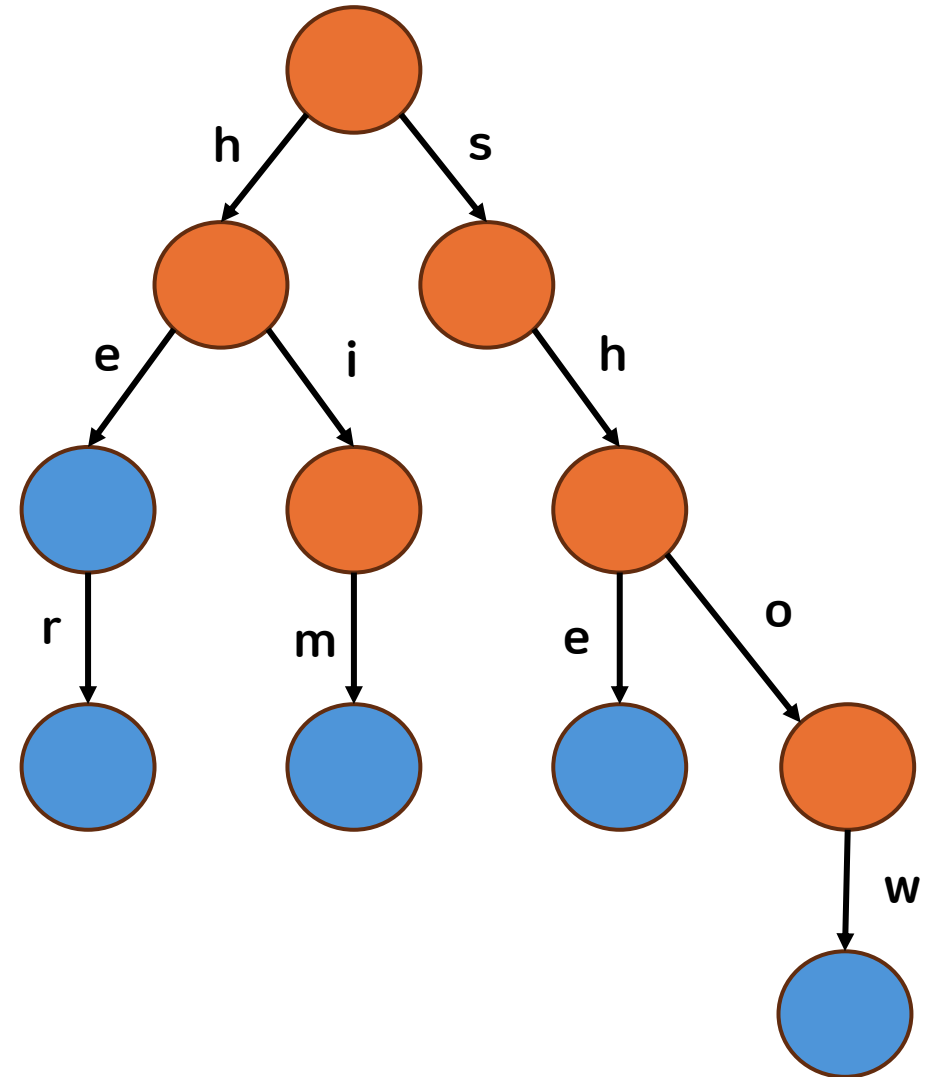


접두사 찾기 /14426

he, she, her, him, show

어떤 문자열의 접두사가
다른 문자열 자체일 수도 있으므로

문자열 자체의 정보가 필요하면
그 정점이 문자열에 대응되는지에
대한 정보를 따로 넣어줘야 함



접두사 찾기 /14426

한 문자열당 정점은 최대
문자열의 개수만큼 생길 수 있음

즉 최대 $N * L$ 개의 정점이 생김
-> 메모리를 많이 먹음

insert, exist 연산 모두 $O(L)$

-> $O((N + M) * L)$

$N, M = 10,000, L = 500$ 이므로 통과 가능

```
int adj[MAX][26], cnt;
bool chk[MAX];

void insert(const string& s){
    int state = 0;
    for(auto& i : s){
        if(!adj[state][i - 'a']) adj[state][i - 'a'] = ++cnt;
        state = adj[state][i - 'a'];
    }

    chk[state] = 1;
}

bool exist(const string& s){
    int state = 0;
    for(auto& i : s){
        if(!adj[state][i - 'a']) return 0;
        state = adj[state][i - 'a'];
    }

    return 1;
}
```

질문?

전화번호 목록 / 5052

백준 5052 / <https://www.acmicpc.net/problem/5052>

N개의 문자열이 주어짐

N개의 문자열에 대해서 모든 문자열이
다른 문자열의 접두사가 아니면 YES, 아니면 NO를 출력

전화번호 목록 / 5052

예제 입력 1 복사

```
2
3
911
97625999
91125426
5
113
12340
123440
12345
98346
```

예제 출력 1 복사

```
NO
YES
```

전화번호 목록 / 5052

Case 1

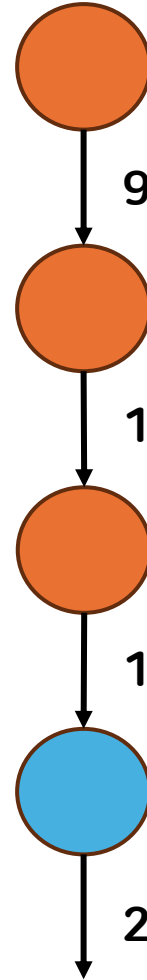
기존 번호가 새 번호의 접두사인 경우

기존: 911

삽입: 91125426

새 번호를 따라 내려가는 도중
chk == true인 노드를 만나면

기존 번호가 새 번호의 접두사
-> NO 출력



전화번호 목록 / 5052

Case 2

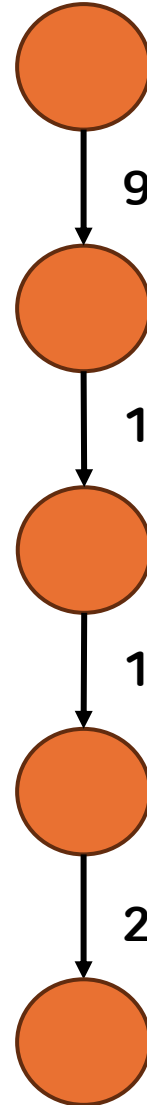
새 번호가 기존 번호의 접두사인 경우

기존: 91125426

삽입: 911

삽입이 끝난 노드 아래에
이미 이어지는 간선이 존재하면

새 번호가 기존 번호의 접두사
-> NO 출력



전화번호 목록 / 5052

각 번호를 하나씩 트라이에 삽입한다

1. 이동 중 $chk == 1$ 인 정점을 만나면 NO
2. 마지막 정점에 이미 자식이 있으면 NO
3. 모든 삽입이 끝나면 YES

전화번호 길이의 합을 S 라고 하면
시간 복잡도 $O(S)$, 공간 복잡도 $O(S)$

질문?

개미굴 / 14725

백준 14725 / <https://www.acmicpc.net/problem/14725>

N개의 문자열 리스트가 주어짐

각 문자열 리스트를 트라이에 삽입 할 때
트라이의 구조를 출력

개미굴 / 14725

예제 입력 1 복사

```
3
2 B A
4 A B C D
2 A C
```

예제 출력 1 복사

```
A
--B
----C
-----D
--C
B
--A
```

개미굴 / 14725

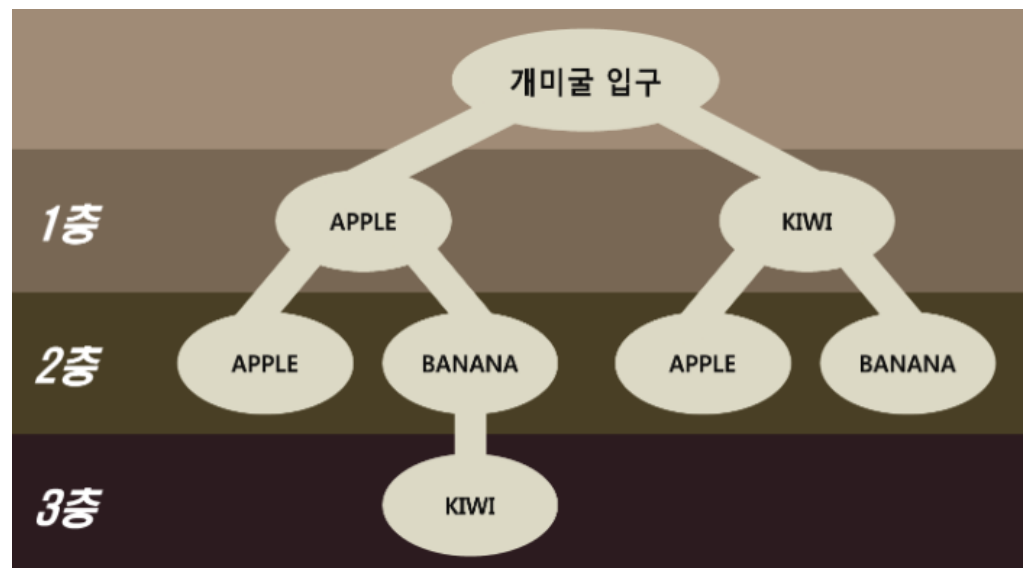
기존 문제

문자열 하나를 문자 단위로 삽입

개미굴

문자열 리스트를 문자열 단위로 삽입

- KIWI BANANA
- KIWI APPLE
- APPLE APPLE
- APPLE BANANA KIWI

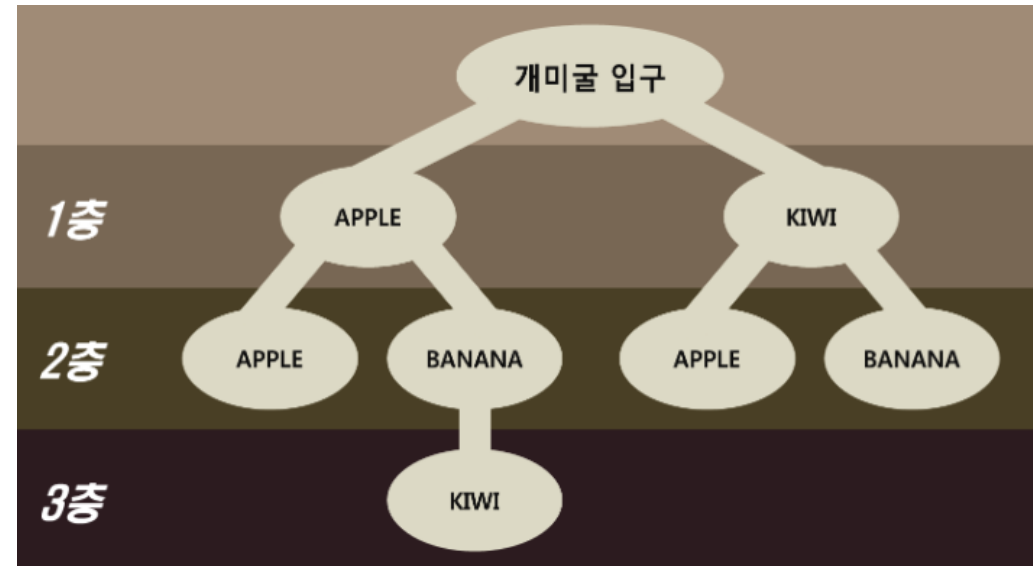


개미굴 / 14725

루트부터 DFS로 내려가면서 출력

현재 깊이가 d라면
먹이 이름 앞에 "--"를 d번 출력

출력 순서는 사전순이어야 하므로
각 노드의 자식을 정렬된 순서로 순회



```
APPLE
--APPLE
--BANANA
----KIWI
KIWI
--APPLE
--BANANA
```

개미굴 / 14725

먹이 이름의 종류가 고정되어 있지 않으므로
간선의 자료구조로 배열이 아닌 map 사용

map을 사용 했으므로 사전 순 정렬이 보장됨

전체 문자열 개수의 총합을 S 라 할 때
시간 복잡도 $O(S \log S)$

$S = 15,000$ 이므로 통과 가능

개미굴 / 14725

```
int cnt;
map <string, int> adj[MAX];
void insert(vector <string>& vec){
    int state = 0;
    for(auto& i : vec){
        if(!adj[state].count(i)) adj[state][i] = ++cnt;
        state = adj[state][i];
    }
}

void dfs(int state, int depth = 0){
    for(auto& [st, nxt] : adj[state]){
        for(int i = 1; i <= depth; i++) cout << "--";
        cout << st << "\n";
        dfs(nxt, depth + 1);
    }
}

void run(){
    cin >> n;
    for(int i = 1; i <= n; i++){
        vector <string> vec;
        int sz; cin >> sz;
        for(int j = 1; j <= sz; j++){
            string st; cin >> st;
            vec.push_back(st);
        }

        insert(vec);
    }

    dfs(0);
}
```

질문?

두 수 XOR / 13505

백준 13505 / <https://www.acmicpc.net/problem/13505>

N개의 수가 주어지고,
XOR한 값이 가장 큰 두 수의 XOR한 결과를 출력

두 수 XOR / 13505

같은 비트끼리 XOR하면 0

다른 비트끼리 XOR하면 1

XOR 결과를 크게 만들려면

높은 자리 비트부터 1을 만드는 것이 이득

$$1000(2) = 8$$

$$0111(2) = 7$$

두 수 XOR / 13505

숫자를 이진수 문자열로 보기

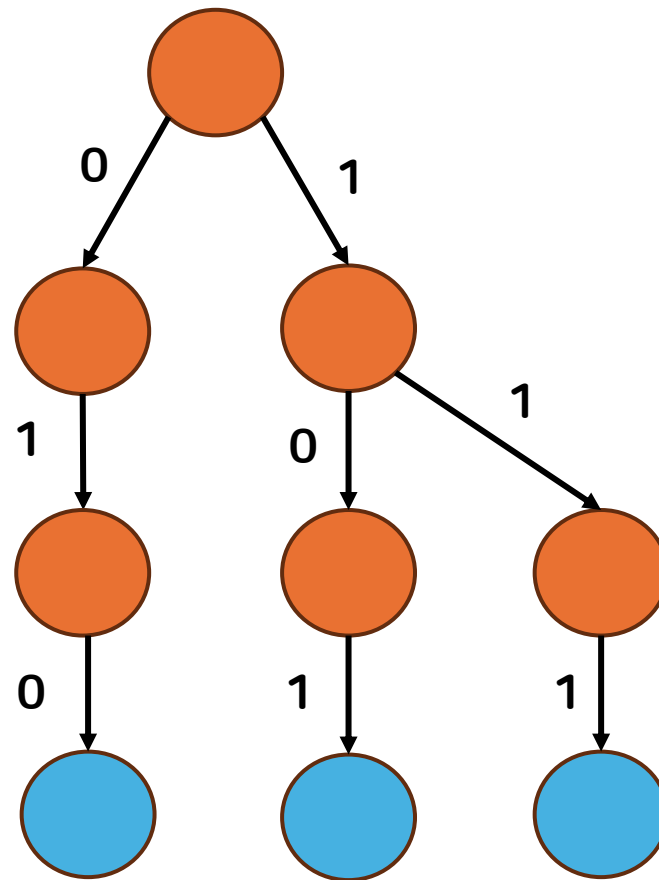
숫자도 고정 길이의 0/1 문자열로 볼 수 있음

5 = 101

2 = 010

7 = 111

각 수를 높은 비트부터 트라이에 삽입
간선의 종류는 0과 1 두 개뿐



두 수 XOR / 13505

어떤 수와 XOR이 큰 상대 찾는법

현재 수 x 의 i 번째 비트가 b 라고 하자

XOR의 i 번째 비트를 1로 만들려면
상대 수의 i 번째 비트는 $!b$ 여야 함

따라서 높은 비트부터 반대 비트가 존재하면
그쪽으로 이동 없으면 같은 비트로 이동

두 수 XOR / 13505

그리디가 맞는 이유?

이진수에서는 높은 자리 비트가
낮은 자리 전체보다 큼

i번째 비트를 1로 만들 수 있다면
그 선택을 포기하고 아래 비트를 아무리 잘 만들어도
더 큰 값을 만들 수 없음

그래서 높은 비트부터 독립적으로 결정해도 됨

두 수 XOR / 13505

1. 모든 수를 0/1 트라이에 삽입
2. 각 수에 대해 XOR이 최대가 되는 상대를 탐색
3. 그 중 최댓값을 답으로 사용

수의 범위가 10^9 이하라면
30번 비트부터 0번 비트까지 확인하면 충분

시간 복잡도 $O(31N) \rightarrow O(N)$

두 수 XOR / 13505

```
void insert(int num){
    int state = 0;
    for(int i = 30; i >= 0; i--){
        bool b = num & (1 << i);
        if(!adj[state][b]) adj[state][b] = ++cnt;
        state = adj[state][b];
    }
}

int max_xor(int num){
    int state = 0, ret = 0;
    for(int i = 30; i >= 0; i--){
        bool b = num & (1 << i);
        if(!adj[state][!b]){
            state = adj[state][b];
        }
        else{
            state = adj[state][!b];
            ret += 1 << i;
        }
    }

    return ret;
}

void run(){
    cin >> n;
    for(int i = 1; i <= n; i++){
        cin >> a[i];
        insert(a[i]);
    }

    int result = 0;
    for(int i = 1; i <= n; i++) result = max(result, max_xor(a[i]));

    cout << result;
}
```

질문?

고생하셨습니다